

[Home](#)[GitHub](#)

CLASSES

[ArticlesService](#)[ErrorBoundary](#)

EVENTS

[SIGINT](#)[unhandledRejection](#)

NAMESPACES

[apiConfig](#)[contactoEmergenciaService](#)[diarioService](#)

GLOBAL

[404_Error_Handler](#)[ACTIVIDADES](#)[API_URL](#)[ASPECTOS_COGNITIVOS](#)[ActionCard](#)[ActivitySelector](#)[App](#)[Articulos](#)[AuthButtons](#)[AuthLayout](#)[Button](#)[CORS_Configuration](#)[Card](#)[Carousel](#)[Checkbox](#)[CircularProgress](#)[CognitionSelector](#)[Diario](#)[DiaryBanner](#)[DiaryEditor](#)[DiaryEntry](#)[EMOCIONES](#)[EmergencyButton](#)[EmergencyModal](#)[EmotionSelector](#)[EmotionStats](#)

backend/src/controllers/diario.controller.js

```
1.  /**
2.   * @file Controlador para las entradas del diario.
3.   * @description Gestiona las operaciones CRUD para las entradas del
4.   * @requires ../models/diario_mongoose
5.   */
6.  const Diario = require('../models/diario_mongoose');
7.
8.  /**
9.   * @function crearEntradaDiario
10.  * @description Crea una nueva entrada en el diario para el usuario
11.  * @param {object} req - Objeto de petición de Express.
12.  * @param {object} res - Objeto de respuesta de Express.
13.  * @returns {Promise<void>}
14.  */
15.  exports.crearEntradaDiario = async (req, res) => {
16.    try {
17.      const { titulo, cuerpo, password } = req.body;
18.      const usuarioId = req.user.id;
19.
20.      // Validar campos requeridos
21.      if (!titulo || !cuerpo) {
22.        return res.status(400).json({
23.          message: 'Título y cuerpo son requeridos'
24.        });
25.      }
26.
27.      // Crear objeto de entrada
28.      const entradaData = {
29.        usuarioId,
30.        titulo,
31.        cuerpo
32.      };
33.
34.      // Añadir password solo si se proporcionó
35.      if (password && password.trim() !== '') {
36.        entradaData.password = password;
37.      }
38.
39.      const nuevaEntrada = new Diario(entradaData);
40.      await nuevaEntrada.save();
41.
42.      // No devolver el password hashado en la respuesta
43.      const entradaResponse = nuevaEntrada.toObject();
44.      delete entradaResponse.password;
45.
46.      res.status(201).json({
47.        message: 'Entrada del diario creada con éxito',
48.        entrada: entradaResponse
49.      });
50.    } catch (error) {
51.      console.error('Error al crear entrada del diario:', error);
52.      res.status(500).json({
```

```

53.         message: 'Error al crear la entrada del diario',
54.         error: error.message
55.     });
56. }
57. };
58.
59. /**
60.  * @function obtenerEntradasDiario
61.  * @description Obtiene todas las entradas del diario del usuario au
62.  * @param {object} req - Objeto de petición de Express.
63.  * @param {object} res - Objeto de respuesta de Express.
64.  * @returns {Promise<void>}
65.  */
66. exports.obtenerEntradasDiario = async (req, res) => {
67.     try {
68.         const usuarioId = req.user.id;
69.
70.         // Buscar entradas del usuario, ordenadas por fecha (más rec
71.         const entradas = await Diario.find({ usuarioId })
72.             .select('-password') // No devolver la contraseña hashea
73.             .sort({ createdAt: -1 }); // Ordenar por fecha descender
74.
75.         res.status(200).json(entradas);
76.     } catch (error) {
77.         console.error('Error al obtener entradas del diario:', error
78.         res.status(500).json({
79.             message: 'Error al obtener las entradas del diario',
80.             error: error.message
81.         });
82.     }
83. };
84.
85. /**
86.  * @function obtenerEntradaDiarioPorId
87.  * @description Obtiene una entrada específica del diario por su ID.
88.  * @description Si el solicitante es el propietario, se le da acceso
89.  * @description Si la entrada es pública (con contraseña), se verifi
90.  * @param {object} req - Objeto de petición de Express.
91.  * @param {object} res - Objeto de respuesta de Express.
92.  * @returns {Promise<void>}
93.  */
94. exports.obtenerEntradaDiarioPorId = async (req, res) => {
95.     try {
96.         const { id } = req.params;
97.         const { password } = req.body;
98.         const usuarioId = req.user ? req.user.id : null;
99.
100.         const entrada = await Diario.findById(id);
101.
102.         if (!entrada) {
103.             return res.status(404).json({ message: 'Entrada del diar
104.         }
105.
106.         // Si el usuario es el propietario, puede acceder
107.         if (entrada.usuarioId.toString() === usuarioId) {
108.             return res.status(200).json(entrada);
109.         }
110.
111.         // Si la entrada no tiene contraseña, es privada para el aut
112.         if (!entrada.password) {
113.             return res.status(403).json({ message: 'Acceso denegado.
114.         }
115.
116.         // Si la entrada tiene contraseña, verificarla

```

```

117.         if (!password) {
118.             return res.status(401).json({ message: 'Se requiere cont
119.         }
120.
121.         const passwordCorrecta = await entrada.compararPassword(pass
122.         if (passwordCorrecta) {
123.             const entradaPublica = entrada.toObject();
124.             delete entradaPublica.password; // No exponer el hash de
125.             return res.status(200).json(entradaPublica);
126.         } else {
127.             return res.status(403).json({ message: 'Contraseña incor
128.         }
129.
130.     } catch (error) {
131.         res.status(500).json({ message: 'Error al obtener la entrada
132.     }
133. };
134.
135. /**
136.  * @function actualizarEntradaDiario
137.  * @description Actualiza una entrada del diario existente.
138.  * @param {object} req - Objeto de petición de Express.
139.  * @param {object} res - Objeto de respuesta de Express.
140.  * @returns {Promise<void>}
141.  */
142. exports.actualizarEntradaDiario = async (req, res) => {
143.     try {
144.         const { id } = req.params;
145.         const { titulo, cuerpo, password } = req.body;
146.         const usuarioId = req.user.id;
147.
148.         const entrada = await Diario.findById(id);
149.
150.         if (!entrada) {
151.             return res.status(404).json({ message: 'Entrada del diar
152.         }
153.
154.         if (entrada.usuarioId.toString() !== usuarioId) {
155.             return res.status(403).json({ message: 'No tienes permis
156.         }
157.
158.         if (titulo) entrada.titulo = titulo;
159.         if (cuerpo) entrada.cuerpo = cuerpo;
160.         if (password) {
161.             entrada.password = password;
162.         } else if (password === '') { // Permitir eliminar la contra
163.             entrada.password = undefined;
164.         }
165.
166.
167.         await entrada.save();
168.         res.status(200).json({ message: 'Entrada del diario actualiz
169.     } catch (error) {
170.         res.status(500).json({ message: 'Error al actualizar la entr
171.     }
172. };
173.
174. /**
175.  * @function eliminarEntradaDiario
176.  * @description Elimina una entrada del diario.
177.  * @param {object} req - Objeto de petición de Express.
178.  * @param {object} res - Objeto de respuesta de Express.
179.  * @returns {Promise<void>}
180.  */

```

```
181. exports.eliminarEntradaDiario = async (req, res) => {
182.   try {
183.     const { id } = req.params;
184.     const usuarioId = req.user.id;
185.
186.     const entrada = await Diario.findById(id);
187.
188.     if (!entrada) {
189.       return res.status(404).json({ message: 'Entrada del diario no encontrada' });
190.     }
191.
192.     if (entrada.usuarioId.toString() !== usuarioId) {
193.       return res.status(403).json({ message: 'No tienes permiso para eliminar esta entrada' });
194.     }
195.
196.     await Diario.findByIdAndDelete(id);
197.     res.status(200).json({ message: 'Entrada del diario eliminada exitosamente' });
198.   } catch (error) {
199.     res.status(500).json({ message: 'Error al eliminar la entrada' });
200.   }
201. };
```

Documentation generated by [JSDoc 4.0.5](#) on Fri Feb 13 2026 09:07:21 GMT+0000 (Coordinated Universal Time) using the [docdash](#) theme.